

TIME/SPACE

$O(g(n)) = f(n) \Rightarrow \forall n \geq n_0, (f(n) \leq c \cdot g(n)) \quad n_0 > 1$
 $\Omega(g(n)) = f(n) \Rightarrow \forall n \geq n_0, (f(n) \geq c \cdot g(n)) \quad n_0 > 1$
 $\Theta(g(n)) = f(n) \Rightarrow \forall n \geq n_0, (c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n))$

Primitives: var, ass., pointer follow, arithmetic, return, method call, array element access

Unfolding example: $T(n) = c_1 + 2T(n-1)$
 $T(0) = c_0$
 $T(n) = 3c_1 + 2T(n-2)$
 $T(n) = 2^{k-1}c_1 + 2^k T(n-k)$ subs. $k=n$
 $T(n) = (2^n - 1)c_1 + 2^n c_0 \Rightarrow O(2^n)$

Recursion types: linear: 1 call per
 Binary: 2 or more calls per
 Multiple: 3 or more calls per

Space: count aux space, each stack frame stores a pointer
 loops $O(1)$ if space reused between iterations

SEQUENCES

method	Array	SLL	CLL	DLL
get(i)	$O(1)$	$O(n)$	$O(n)$	$O(n)$
set(i, v)	$O(1)$	$O(n)$	$O(n)$	$O(n)$
remove	$O(n)$	$O(1)$	$O(1)$	$O(1)$
First()	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Last()	$O(1)$	$O(n)$	$O(n)$	$O(1)$
addFirst()	$O(n)$	$O(1)$	$O(1)$	$O(1)$
addLast()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
first()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
last()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
insert(i)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
remove(i)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
size()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
rotate()	$O(n)$	$O(n)$	$O(1)$	$O(1)$
concat()	$O(n)$	$O(1)$	$O(1)$	$O(1)$

Queue with array: all $O(1)$
 $head = (head + 1) \bmod \text{arr.length}$
 $tail = (head + 2) \bmod \text{arr.length}$

Queue with LLs: all $O(1)$
 remove at head, add to tail

Queue methods: enqueue, dequeue, first, size, empty

Stacks: all $O(1)$, arr or LL
 methods: push, pop, size, peek

Deque: addFirst, addLast, first, removeFirst, removeLast, last, size

Array implementation: all $O(1)$
 $head = (head - 1 + N) \bmod N$

DLL is a deque, all $O(1)$

Amortised formulae: $\sum_{k=0}^n k = \frac{n(n+1)}{2}$
 $\sum_{k=0}^n k^2 = \frac{n(n+1)(n+2)}{6}$
 $\sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4}$
 $S_n = \frac{n(n+1)}{2}$

LISTS

List ADT: add(i, e), remove(i)
 set(i, e), get(i), size, isEmpty

Array: all $O(1)$ except add and remove

Dynamic Array List: resizing by 2 makes amortised $O(1)$:
 $O(2^i)$ after $2^{i-1} - 1$ $O(1)$ s

Positional lists: positions inmut.
 first, last, before(p), after(p):
 return positions

isEmpty, size, addFirst, addLast, addAfter(p, e), addBefore(p, e), set(p, e), remove(p)

DLL but nodes encapsulated

Array: store p AND e, $O(n)$ ins/del

ITERATORS

hasNext, next, optional remove

Snapshot: make copy, iterate, $O(n)$ time/space

Lazy: iterate, $O(1)$ space, fail-fast if the og changes

TREES

nonEmpty: has root, all non roots have parent

a is ancestor of b if a=b or a is ancestor of parent of b

a is ancestor of b $\Rightarrow$ b is descendant of a

non-leaves are internal

root(), parent(p), children(p), numChildren(p), isInternal(p), isExternal(p), isRoot(p), size, isEmpty

Depth: root: 0, node: depth(parent)+1

Height: leaf: 0, node: height of highest child + 1, both $O(n)$

height(root) = depth(lowest tree)

Binary tree: left(p), right(p), sibling(p)

Level d: all nodes with depth=d

Max # nodes at d = 2^d

Given proper BT, next = next+1

Linked BT: nodes store parent, element, left/right, ins/del $O(1)$, depth, height $O(n)$

Array BT: index root=0
 left of p = $2p+1$, right = $2p+2$
 time same, reserved space = 2^{n+1}

TRAVERSALS

Pre: root $\rightarrow$ left $\rightarrow$ right

In: left $\rightarrow$ root $\rightarrow$ right

Post: left $\rightarrow$ right $\rightarrow$ root

BFS: level by level:
 enqueue(p)
 while queue empty
 p = dequeue
 visit p
 for c: children(p)
 enqueue c

PRIORITY QUEUES

ADT: add(k, v), min(), remove(), size, isEmpty

Impl.	Insert	Remove	Min
Sorted	$O(n)$	$O(1)$	$O(1)$
Unsorted	$O(1)$	$O(n)$	$O(n)$
Heap	$O(\log n)$	$O(\log n)$	$O(1)$

HEAPS

Order: children key > parent key
 Complete BT: $\dots$, heights $\leq \log(n)$

Insert: up-heap bubbling
 if key(child) < key(parent)
 invert, continue
 else
 Oke ;, worst case $O(\log n)$

Remove: down-heap bubbling
 Remove root, replace with last added node. Then, k_c -lowest key child, if $k_{root} > k_c$, invert, bubble down, worst case $O(\log n)$

Array: index f(p) of node p:
 root: f(p) = 0
 left(q): f(p) = f(q) + 1
 right(q): f(p) = f(q) + 2

DLL: last pos: tree path (binary)
 Left 0, Right 1, highest val $\Rightarrow$ last
 left: $\lfloor \log_2 n \rfloor$ bits, new: $\lfloor \log_2(n+1) \rfloor$ bits
 Both $O(\log n)$ for insert, remove

Bottom-up construction:
 1. n nodes $\rightarrow$ take $\frac{n+1}{2}$ nodes, create $\frac{n+1}{2}$ trees with only root
 2. Take $\frac{n+1}{4}$ nodes, root for prev trees, trivial order of children, compare child to parent and swap if necessary
 3. Keep going, # of new trees at iter. i = f(i) = $\frac{n+1}{2^i}$, $O(n)$ total

APQs: remove(a), change Key(e, k), change Value(e, v). Lists inefficient

Arrays: third element per entry, index in underlying array. ^{other wise} $O(\log n)$ for remove and change Key, $O(1)$

SORTING

Selection: unsorted list PQ, remove min key, insert it as $O(1)$, removals $O(n)$, total $O(n^2)$

In-place: partition array in 2, sorted (one element at start), and unsorted. Find smallest element in unsorted, move to start of unsorted, expand sorted to this new element

Insertion: add to sorted list PQ, remove to sort.
 Add new element to end, swap until next has lower or equal key
 Takes $O(n^2)$, I = # of swaps

In place: 1st is sorted, move to 2nd, swap if out of order, continue over whole array

Heap sort: $O(n)$ bottom-up construction, $O(n \log n)$ otherwise
 $O(n \log n)$ removal, total $O(n \log n)$

In-place: use max-heap, remove max, put at end of array, continue until sorted

Merge sort: divide and conq.

Divide sequence into S_1, S_2 with $\lfloor \frac{n}{2} \rfloor$ and $\lceil \frac{n}{2} \rceil$ elements until sequences with 1 element ($\log \lceil \frac{n}{2} \rceil$ steps). Combine them two at a time, comparing at each step. With array, we keep indices in each array and assign lowest from either to arr[i+j], then increment the index of the array we inserted the element from.

Quick sort: pick pivot p, divide into $L(<p)$, $E(=p)$, $G(>p)$. Recursively sort the 3 subsets, combine back $L+E+G$.
 $O(n^2)$ worst case (sorted, pivot last)
 $O(n \log n)$ randomised avg

In-place: take pivot and l/r indices. If element at L is > p and at r < p, swap them, else L++, r--. When they cross, recurse with L, l, r-1, l, r, l, r, r

Quick select: divide in L, E, G, if $k \in L$, recurse on L, if $k \in E$, return pivot, else recurse on G with $k = k - |E|$.
 $O(n)$ randomised average, worst $O(n^2)$

Lower bound on sorting:
 All comparisons can be made into a decision tree, with n! leaves, so the height will be $\log(n!)$, i.e. the # of comparisons. There are $\frac{n}{2}$ elements $\geq \frac{n}{2}$ in the seq, so $\log(n!) \geq \frac{n}{2} \log(\frac{n}{2})$, which is $\Omega(n \log n)$.

LINEAR SORTING

Bucket: all entries have keys, we iterate and place each entry in a bucket corresponding to key. Make auxiliary array of arrays. Remove each entry and put in end of kth array. Then for each key, for each entry in that bucket, remove it and add to end of S.

Radix: sorting elements with strong key k and weak v, $k_1 < k_2$ or $k_1 = k_2 \wedge v_1 < v_2$. We sort by v first, then k by stable bucket sort sequentially (LSD). For MSD, we can recurse on each k bucket and sort with v. MSD works for variable key amount.

Algo	Stable	Inplace	Worst	Best	Space
SEL	6/7	Yes	$O(n^2)$	$O(n^2)$	$O(1)$
Ins	Yes	Yes	$O(n^2)$	$O(n)$	$O(1)$
Heap	No	Yes	$O(n \log n)$	$O(n \log n)$	$O(1)$
Merge	Yes	No	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick	No	Yes	$O(n^2)$	$O(n \log n)$	$O(\log n)$
Bucket	Yes	No	$O(n+k)$	$O(n+k)$	$O(n+k)$
Radix	Yes	No	$O(d \cdot \log(n))$	$O(d \cdot \log(n))$	$O(n+k)$

MAPS

ADT: size, isEmpty, get(k), put(k, v), remove(k), keySet, values, entrySet, keyPairs

List impl: removal swaps last el. to empty position. All fundamental methods $O(n)$.

Hash tables: keys processed into ints $[0, M-1]$, ideally uniform but collisions possible, so we use array of buckets.

Hash codes: $h(n: Obj) \rightarrow \text{hash.int}$. Split into hash code creation and compression to $[0, M-1]$

CREATION

Bit representation: for 32b types, cast the number to int. For long/float (bits), sum upper and lower (ignore overflow) or XOR bit representations of upper/lower

Polynomial: for strings, we use $a \neq 0$, x_i : bit at position i, $x_0 a^{n-1} + x_1 a^{n-2} + \dots + x_{n-1} a^0 + x_{n-1} = x_{n-1} a + (x_{n-2} a + \dots + (x_1 a + x_0))$

Pick a s.t. it has low-order bits $\neq 0$.

Cyclic-shift: just take n left bits to right

COMPRESSION

Division: hashed N, N prime. Collisions in

MAP: compressed = $((a \cdot \text{hash}(b) \bmod p) \bmod N) \bmod p$, $a, b \in \text{int } [0, p-1]$, $a \neq 0$, p-prime $< N$

COLLISION HANDLING

Separate chaining: bucket array, runs $O(n/N)$, n elements in size N array. $2 = n/N$ load factor, should be < 1 . Space-heavy.

Lin. probing: collisions put elsewhere. Key k, hash $h(k) \rightarrow \text{index} = h(k)$. If occupied, check $(h(k)+1) \bmod N$, then $(h(k)+2) \bmod N$, etc.

On removal, replace with tomb to keep the search chain. Search then skips over tombs and continues, put replaces tombs.

Qued. probing: search $h(k)$ at $(h(k)+f(i)) \bmod N$, $f(i) = i^2$, $i = 1, 2, 3, \dots$. Bad if $n \geq N/2$ or N not prime

If $\lambda > 0.5$ for LP or > 1 for SC, new arr size $\geq 2N$, change compression. $O(N)$ worst

SORTED MAP

ADT: first/lastEntry, ceiling/floorEntry(k), lower/higherEntry(k), subMap(k1, k2)

Ceiling/floor - largest key $\geq k / \leq k$ | subMap: $k_1 < k, k_2 < k_2$

Lower/higher - least key $< k / > k$

Binary search over sorted arr, if success, key returns, else approx location, return left/right element. All ops $O(\log n)$ subMap

$O(s \log n)$, s - time to copy to new map.

Add/remove now $O(n)$, need to left/right shift the keys in lookup array.

